

Machine Learning-based Dispatching for a Wet Clean Station in Semiconductor Manufacturing

Sung-hoon Jeong^a, Gyusun Hwang^b, Ju-Yong Lee^{c, 1*}, and Jun-Hee Han^{a, 2*}

^aDepartment of Industrial Engineering, Pusan National University, Busan 46241, Republic of Korea.

^bSchool of Industrial Engineering, University of Ulsan, Ulsan 44610, Republic of Korea.

^cCollege of Business Administration, Kangwon National University, Chuncheon 24341, Republic of Korea.

Abstract

This study deals with the scheduling problem to minimize makespan at a wet clean station in semiconductor fabrication. The wet clean station is comprised of sequential chemical and rinsing baths for cleaning wafer lots and multiple robot arms for lot handling. In the station, wafer lots are sequentially immersed in several baths for cleaning to eliminate residual contaminants and stains that cause defects on wafer surfaces. The station can process various types of products, and the specific order of immersion differs depending on the product type. Unlike typical scheduling problems, the information required for constructing schedules, such as processing times and sequences inside the station, is not available. The only available data are historical logs that record when each lot was tracked into and out of the station. However, even when cleaning the same product type, the duration that lots spend in the station may vary based on the combination of product types being cleaned simultaneously and the settings of the station. Thus, using the time records, this study proposes a dispatching method based on machine learning models such as multiple linear regression, deep neural network, and convolutional neural network. The proposed algorithms were evaluated and verified by comparing them with dispatching methods used in a semiconductor fab in Korea. Through this experiment, we observed that the proposed models can provide scheduling solutions that are practical and effective in a rapidly changing production setting. These models have the potential to enhance the capacity of a wet clean station.

Keywords: convolutional neural network, deep neural network, machine learning, scheduling, semiconductor manufacturing, wet clean station

^{1*} Co-corresponding author at jy.lee@kangwon.ac.kr

^{2*} Co-corresponding author at junhan@pusan.ac.kr

These authors contributed equally to this work.

Acknowledgment

This work was supported by a project for Smart Manufacturing Innovation R&D funded Korean Ministry of SMEs and Startups in 2023 (Project No. RS-2022-00140540).

Author Statement

Sung-hoon Jeong: Conceptualization, Methodology, Software

Gyusun Hwang: Validation, Data curation, Writing – Original Draft.

Ju-Yong Lee: Conceptualization, Validation, Formal analysis Writing – Original Draft

Jun-Hee Han: Conceptualization, Methodology, Validation, Formal Analysis, Writing – Original Draft, Product administration

Machine Learning-based Dispatching for a Wet Clean Station in Semiconductor Manufacturing

Abstract

This study deals with the scheduling problem to minimize makespan at a wet clean station in semiconductor fabrication. The wet clean station is comprised of sequential chemical and rinsing baths for cleaning wafer lots and multiple robot arms for lot handling. In the station, wafer lots are sequentially immersed in several baths for cleaning to eliminate residual contaminants and stains that cause defects on wafer surfaces. The station can process various types of products, and the specific order of immersion differs depending on the product type. Unlike typical scheduling problems, the information required for constructing schedules, such as processing times and sequences inside the station, is not available. The only available data are historical logs that record when each lot was tracked into and out of the station. However, even when cleaning the same product type, the duration that lots spend in the station may vary based on the combination of product types being cleaned simultaneously and the settings of the station. Thus, using the time records, this study proposes a dispatching method based on machine learning models such as multiple linear regression, deep neural network, and convolutional neural network. The proposed algorithms were evaluated and verified by comparing them with dispatching methods used in a semiconductor fab in Korea. Through this experiment, we observed that the proposed models can provide scheduling solutions that are practical and effective in a rapidly changing production setting. These models have the potential to enhance the capacity of a wet clean station.

Keywords: convolutional neural network, deep neural network, machine learning, scheduling, semiconductor manufacturing, wet clean station

1. Introduction

The production of semiconductors involves many chemical procedures. At the end of a process, there might be contaminants or stains left on the wafer surfaces which can negatively impact electrical performance and result in a decrease in yield. To prevent such quality issues, wafer cleaning makes up 30% of the entire process (Lee *et al.*, 2007). The cleaning is performed in a wet clean station which is designed to remove any residual contaminants on the wafer surfaces. The station is comprised of multiple baths in series, robot arms, and a waiting area for lots before they are moved to the next bath. The baths include chemical baths and rinse baths, the former filled with different chemicals to loosen contaminants, and the latter to rinse off any remaining contaminants. The robot arms handle one lot at a time, by loading and unloading it, and transporting it to the different baths. Fig. 1 illustrates the series of baths in a wet clean station, each filled with a different chemical. In the station, wafer lots are sequentially immersed in several baths required for cleaning to eliminate residual contaminants and stains. The specific order of immersion differs depending on the product type.

One lot is tracked into the wet station at a time, but up to four lots can be processed simultaneously within the station. Accordingly, the station can process different types of products with distinct orders of immersion (called *recipe*). In other words, lots with different recipes are immersed in different baths in different orders, and the immersion time is also different. Hence, some recipes may omit one or more baths. Additionally, even when cleaning the same product type, the duration that lots spend in the station may vary based on the combination of product types being cleaned simultaneously and the settings of the station. The robot arms transporting the wafer lots inside the station are arranged in a series (Kim *et al.*, 2014), so it is impossible to make any alteration to the bath sequence specified in the recipes. This may result in waiting times due to the preceding wafer lots. Therefore, to maximize wet clean station productivity, it is essential to have an efficient scheduling system in place for processing the wafer lots.

This study focuses on the scheduling problem for a wet clean station in semiconductor manufacturing, and the objective function is to minimize the makespan. If the baths inside the wet clean station are regarded as independent machines, the problem can be modeled as a flow shop. Also, in the wet clean station, wafer lots can skip some baths according to their recipes, referred to as *missing operations* in the literature. The considered scheduling problem can be defined as $FS|missing\ operations|C_{max}$ using a three-field notation by Graham *et al.* (1979), where FS and C_{max} represent flow shop and

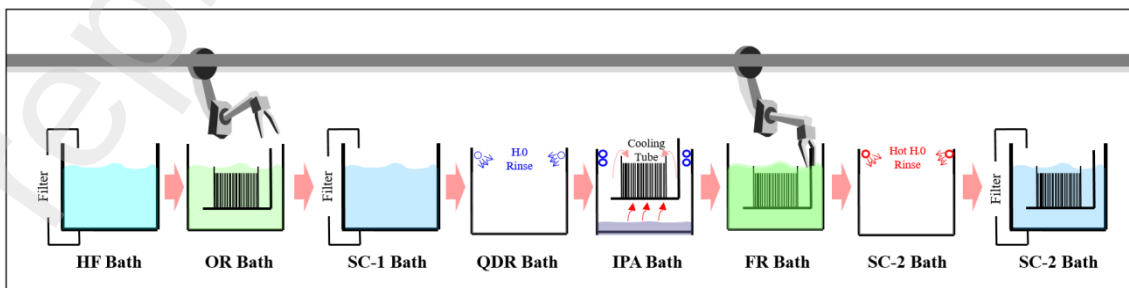


Fig. 1: A graphical view inside a wet clean station with eight baths and two robot arms

makespan, respectively. Note that, if all lots are immersed in at least 3 baths, the complexity of this problem is NP-complete since the classic 3-machine flow shop makespan problem is NP-complete (Garey *et al.*, 1976).

In general, scheduling studies assume that schedules have access to all necessary information for constructing schedules beforehand, including processing times, process sequences, and start and completion times of operations. However, this is not the case for scheduling at the wet clean station in semiconductor manufacturing. First, logs of the order in which lots were immersed in each bath are not provided to the scheduler. Furthermore, the scheduler is not given the times that lots spent in the baths, which are not predetermined but set empirically by an operator and can change during the scheduling period. Although the settings depend on the recipe, it is only stored as parameters in the wet clean station and is not transmitted to the manufacturing execution system (MES) where the scheduler runs. Additionally, as mentioned, even for the same recipe, times may differ based on the station's status, making it impossible to accurately predict the times for scheduling purposes at the wet clean station.

In general, the MES communicates with the tool controller (TC) that collects data inside equipment and receives it. However, the TC inside the wet clean station does not collect the internal data including immersion operations and parameters. This results in the MES having no information on the internal processes of the wet clean station. Hence, we do not have information on the start and end times of bath operations or the waiting time of wafer lots for specific baths. It is also not possible to analyze any loss caused by recipe changes. The only information transmitted to the MES is the loading and unloading times of lots at the wet clean station. Therefore, the classic scheduling approach, which assumes all information is available, is not applicable for the considered wet clean station. Due to these issues, very simple methods such as using the same recipe continuously or 'first come first served' (FCFS) are used to operate the wet clean station in actual fabs. This feature is also found in other processes such as implantation, chemical mechanical polishing (CMP), and several real-world problems (Lee and Kim, 2021).

Instead of traditional scheduling methods, this study developed scheduling algorithms based on machine learning models: multiple linear regression (MLR), deep neural network (DNN), and convolutional neural network (CNN). The machine learning models aim to predict a time that varies depending on the combinations of recipes loaded sequentially and processed together in the wet clean station. Then, using the prediction results, we can determine the best recipe combination and lot to load next. Our proposed algorithms were evaluated by comparison with dispatching methods used in a Korean semiconductor fab. This study has the potential to enhance the utilization and production capacity of wet clean stations.

The remainder of this paper is organized as follows. Section 2 introduces related studies and their limitations, and Section 3 describes the problem in detail and gives a notation for the suggested models. In Section 4, we develop four machine learning models: a multiple linear regression, a deep neural network, and two types of convolutional neural networks. In Section 5, we explain the proposed simulator and a method for data generation, and we examine the performance of the four proposed

algorithms through computational experiments. Finally, Section 6 discusses and concludes this paper with possible extensions of the research.

2. Related studies and their limitations

Owing to the importance of semiconductor manufacturing, scheduling research has been actively conducted since Wein (1988). The lot scheduling of equipment in a fab has been continuously studied by researchers like Johri (1989) as the semiconductor industry has grown. Scholl and Domaschke (2000) introduced a simulation model for wet etch and furnace operations subject to time constraints. Kim *et al.* (2008) suggested a due-date-based algorithm for lot-order assignment, while Govind *et al.* (2008) proposed real-time scheduling and dispatching methods. Yurtsever *et al.* (2009) developed a heuristic algorithm for efficient diffusion operations. Klemmt and Monch (2009) suggested a list scheduling method and a decomposition approach for a flow shop scheduling problem with a time constraint for consecutive processes. Han and Cho (2015) proposed a two-stage real-time dispatcher integrated with a linear programming technique in photolithography. Moreover, Chung *et al.* (2019) suggested a mathematical model and a heuristic algorithm for two identical parallel machines under consideration for a molding process. Yu and Han (2021) considered flow shops with preventive machine maintenance and suggested an algorithm to find an optimal solution within polynomial time, while Jeong *et al.* (2021) developed various metaheuristics for a flow shop with urgent jobs and limited wait-time constraints. Wu *et al.* (2022) proposed a dynamic programming model and a genetic algorithm for diffusion furnaces in semiconductor fabrication.

Focusing on the operations inside the equipment, there have been several previous studies on cluster tools. Lee *et al.* (2008) reviewed scheduling theories and methods for cluster tools in semiconductor manufacturing. Also, Lee *et al.* (2008) and Kim *et al.* (2014) considered wet clean station scheduling problems with a branch-and-bound algorithm and with a Petri net, respectively. In addition, Kim *et al.* (2014) focused on the inside of the wet clean station, which is also considered in our study, and considered several types of recipes with different bath usage. However, those two studies only focused on the movement sequence of robot arms in the station, and they assumed all information is predetermined and given in advance, which is not always true in real-world situations. In addition, Jung *et al.* (2013) and Kim *et al.* (2015) proposed branch-and-bound algorithms for scheduling problems of cluster tools. Yu *et al.* (2018) focused on a single-armed cluster tool with consideration of chamber cleaning operations, while Pang *et al.* (2018) suggested a simulated annealing algorithm for a bi-objective scheduling problem in a wet clean station. However, they did not consider the inside of the equipment, such as baths and arms.

In this study, we introduce machine learning algorithms for obtaining meaningful information from unclear historical manufacturing operation data. There were a few studies to solve scheduling or dispatching problems by using machine learning algorithms. First, Porioire *et al.* (2006) proposed dispatching rules by using several machine learning algorithms for flexible manufacturing systems. Baly and Hajj (2012) used a support vector machine algorithm for wafer classification, and Susto *et al.* (2014)

and Florian *et al.* (2021) proposed machine learning models for predictive maintenance in manufacturing areas. Bergmann *et al.* (2016) suggested data mining and supervised machine learning methods to emulate job scheduling decisions. There are also several studies using reinforcement learning (RL) models. Wang and Usher (2005) suggested RL for single-machine scheduling and job shop scheduling problems. Shahrabi *et al.* (2017) used RL to enhance the performance of the scheduling method. Shiue *et al.* (2018) proposed real-time dispatching rules by using RL for a smart factory, and Lin *et al.* (2019) adjusted a deep Q network, which combines deep learning and RL, with an edge computing framework to solve job shop scheduling problems. Lee and Kim (2021) used RL for robotics flow shop scheduling problems with variations in processing times within a given time interval. Cho *et al.* (2022) suggested RL with a pointer network for a permutation flow shop makespan problem and the RL showed superior performance to well-known (meta)heuristics. Oh *et al.* (2022) proposed RL for a flexible job shop problem with high variability. As a result of the above literature survey, this study is the first attempt to solve a scheduling or dispatching problem with unclear historical data on processing for wet clean station equipment that requires bath configuration.

3. Problem Description

This section describes the scheduling problem for a wet clean station and provides notation for clear descriptions throughout this paper. This study considers a single wet clean station with eight baths arranged in series, as shown in Fig. 1, and each bath has a different chemical. Wafer lots to be cleaned are loaded one at a time, but up to four lots can be processed simultaneously within the station. When one lot is unloaded from the station, one of the waiting lots can be loaded. Because the baths are arranged in a row, the succeeding wafer lot cannot overtake the preceding wafer lot. Wafer lots that have been loaded are immersed in the baths specified in their recipes. However, as previously mentioned, the scheduler only has the times when wafer lots were tracked into and out of the station.

The purpose of this study, therefore, is the development of machine learning (ML) models for dispatching wafer lots to the wet clean station. Learning the available logs, the ML models predict a time that depends on the combinations of recipes processed simultaneously in the wet clean station. Based on the prediction results, we can find the best recipe to load next for minimizing the makespan which is the completion time of the last processed lot. As previously mentioned, the wet clean station can accommodate up to four wafer lots and process them sequentially. Thus, minimizing the makespan can be achieved with an efficient recipe sequence. Herein, to make an efficient recipe sequence, we define a *set* of recipes, with its size indicated as *NS*. For example, if a set consists of {1, 2, 5, 1}, the first recipe to be loaded in is recipe 1, followed by recipes 2, 5, and 1. That is, a set represents a recipe permutation. In this study, the most important decision is how to compose sets and make an order of the recipes in each set.

We provide a notation for clear descriptions and several basic ideas to develop the algorithms.

Indices and parameters

i	index of a set of wafer lots
NS	number of lots in a set (= size of a set)
j	index of processing order in a set, $j = 1, 2, 3, \dots, NS$
k	index of a recipe, $k=1, 2, 3, \dots, K$
P_k	processing time for recipe k
AP_i	actual time to process lots in set i
OP_i	overlapped processing time of lots in set i
TL_k	total number of lots with recipe k to be scheduled

Variable

$x_{ijk} = 1$ if a lot with recipe k is sequenced in the j th order in set i ; 0, otherwise

In general scheduling problems, the processing time for recipe k (P_k) is a parameter. However, it is not a predetermined value in this study. Therefore, instead of an exact processing time, P_k is estimated as the minimum processing time of recipe k in the historical data. Since four wafer lots can proceed simultaneously inside the wet clean station, the actual time to process the four lots is less than the sum of the processing times of the four lots, as seen in the example in Fig. 2. Hence, we define OP_i as the overlapped processing time of set i , which is given in Equation (1):

$$OP_i = \left(\sum_j^J \sum_k^K P_k \times x_{ijk} \right) - AP_i \quad (1)$$

OP_i represents the reduced processing time by combining and sequencing the lots into a set. In other words, it can make a positive impact on minimizing the makespan, and hence, a high OP_i indicates that set i is good. Therefore, OP_i can be an index to evaluate set i . By making sets to maximize OP_i values, we intend to increase the throughput of the wet clean station and minimize the makespan.

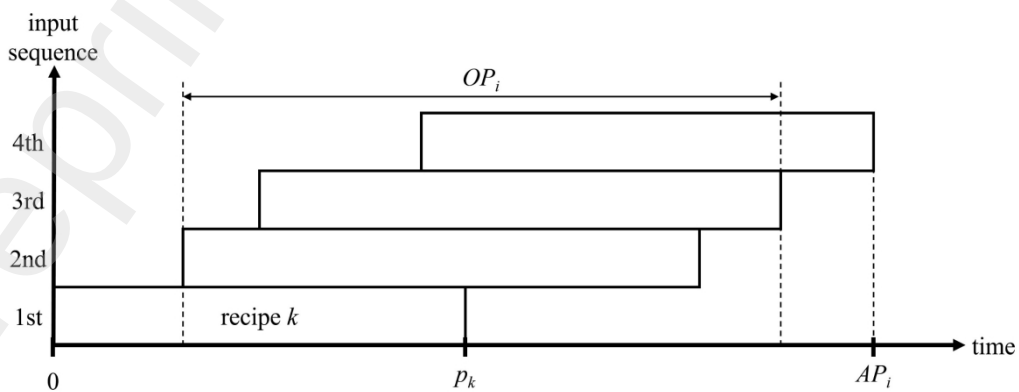


Fig. 2: An example of AP_i , OP_i , and p_k of set i with $NS = 4$

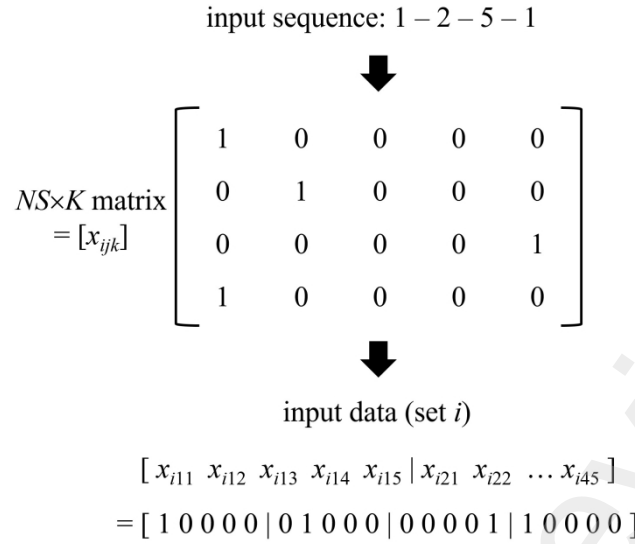


Fig. 3: An example of input data from set i when $NS=4$ and $K=5$

4. Machine Learning Models

In this section, we suggest the use of machine learning models for the scheduling problem of the wet clean station. Our approach involves employing supervised learning algorithms, a subset of machine learning that aims to infer a function from training data. The goal is to comprehend the relationship between the input features and the target prediction output by learning the training data, allowing us to make predictions for new data inputs based on these relationships. To facilitate this process, we first present a data structure for generating training data for ML models, and then suggest a multiple linear regression model and three high-dimensional models.

4.1. Input Data Structure

For constructing learning data, we first need to define an input data structure. A data set for training ML models has to include the recipe sequences of lots that have been tracked into the wet clean station. Thus, we construct an $NS \times K$ matrix, $M = [x_{ijk}]$, for set i . In the matrix, the j th column represents the j th sequence in set i , and the k th row indicates recipe k . Since the input to the wet clean station is done one by one, only one element in each column vector can be (and should be) 1; otherwise, it is 0. After constructing the matrix, we convert it into a single row vector with a size of $1 \times (NS \times K)$ by connecting columns. Fig. 3 represents an example of an input data structure for a set with four lots and five recipes. That is, assuming there are five recipes ($K = 5$), the input sequence is 1, 2, 5, and 1. Then, the matrix with binary elements is squeezed into a single row vector, as shown in the figure.

4.2. Multiple Linear Regression Model

A regression model is a statistical technique that provides a function for investigating and modeling the relationship between one or more independent variables and a dependent variable. In this study, x_{ijk} is an independent variable, and OP_i is a dependent variable, as shown in Fig. 4, with its regression equation expressed in (2) below. The multiple linear regression (MLR) model has a structure in which the input layer is directly connected to the output layer. In this structure, a linear regression function on the input data is learned.

$$OP_i = \beta_0 + \beta_1 x_{i11} + \beta_2 x_{i12} + \beta_3 x_{i13} + \dots + \beta_{NS \times K} \cdot x_{i, NS, K} + \varepsilon \quad (2)$$

To find the β 's in (7), we use a gradient descent algorithm using gradients to minimize the value of a loss function that defines the difference between the predicted value and the actual value. After transforming processing history data into an input structure as mentioned in Section 4.1, regression model (7) is trained via input data sets. For finding each β , we define *cost* as $\frac{1}{m} \sum_{i=1}^m (H(x_{ijk}) - y^{(i)})^2$, where $H(x_{ijk})$ and $y^{(i)}$ are the hypothetical and actual object values, respectively. By using the gradient descent algorithm, we optimize cost and obtain the β 's. After training the MLR model, expected values of OP_i are obtained for all possible combinations of recipes.

4.3. High-dimensional Solve Model

In a high-dimensional solve model, the regression function can have multi-dimensional characteristics by adding hidden layers between input and output layers to form a neural network. The model is designed for modeling a deep neural network and convolutional neural networks. Fig. 5 shows the neural network structure of the suggested high-dimensional solve models.

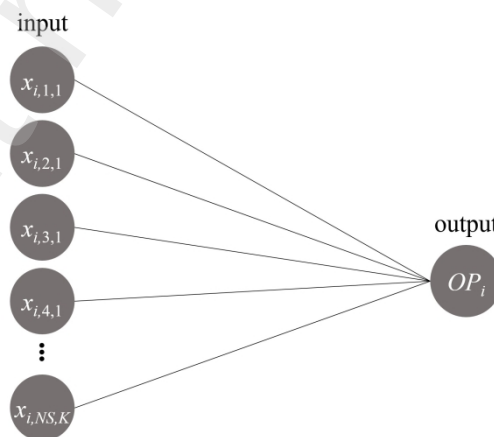


Fig. 4: A graphical example of a multiple linear regression model

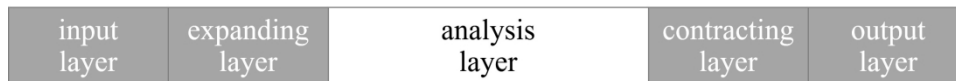


Fig. 5: Structure of layers for the high-dimensional solve model

The structure consists of an input layer, an output layer, and three hidden layers (the expanding layer, the analysis layer, and the contracting layer). Input and output layers are the same as those in the MLR model. The expanding layer increases the number of nodes via linear operations of all nodes in the input layer. Mapping nodes to a higher dimension helps reduce the possibility of data loss Benardos and Vosniakos (2006), so the nodes in the input layer are expanded to 2,048 nodes in the expanding layer. Analysis layers consist of models with a relationship between nodes or feature maps by using weighted values. Therefore, composing effective analysis layers is the key to constructing excellent ML models with training data. This study proposes three different analysis layers based on the characteristics of DNN, CNN1d, and CNN3d models. Because of the different structures of analysis layers, we expect the performances of the three models to be different. After the analysis layer, the contracting layer reduces the dimension of nodes to one and provides a single value to the output layer, so we can obtain the estimated OP_i . The following subsections explain the proposed DNN, CNN1d, and CNN3d models in detail.

4.3.1 Deep Neural Network

The deep neural network (DNN) model focuses on the structure of the hidden layers connected in series. All combinations of nodes in two consecutive layers are fully connected. That is, a linear operation with a coefficient and a constant is performed for each connection. This DNN structure is fundamental to machine learning models, and the DNN model suggested in this paper has four hidden layers. Note that the amount of computation increases exponentially as layers are added.

As shown in Fig. 6, the number of nodes in the input layer of the DNN model depends on the input data length, and hence, the input layer consists of $K \times NS$ nodes.

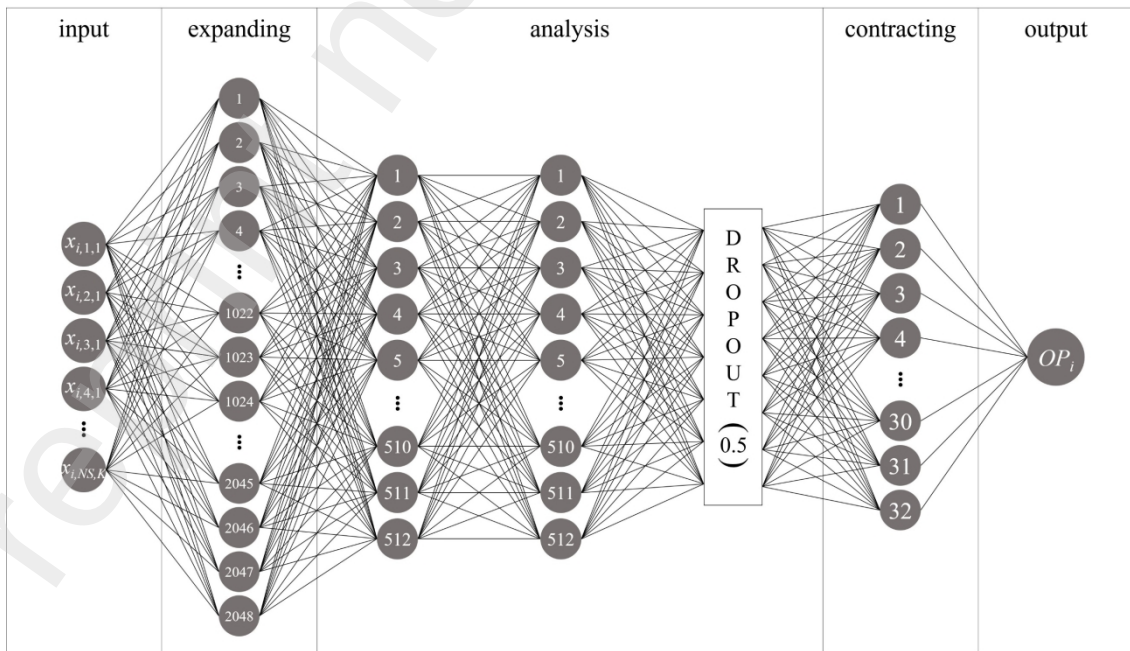


Fig. 6: A graphical example of the deep neural network model

With these nodes, as explained, the expanding layer performs linear operations to increase the number of nodes to 2,048. The analysis layers have two layers with 512 nodes and a dropout layer with a probability of 0.5. Dropout is used to prevent overfitting, and it converts the calculation result to 0 with a probability of 0.5. Then, after the analysis layers, the contracting layer has 32 nodes and executes a linear operator to provide the predicted OP_i to the output layer.

4.3.2 Convolutional Neural Network

A convolutional neural network is a kind of artificial neural network designed to analyze visual data. Inspired by the study of the cat's visual structure, Fukushima (1988) suggested a neocognitron, which is a basic model of a CNN. The CNN structure is divided into a convolution layer and a pooling layer. The convolution layer uses filters to process input data. Like the fully connected structure in a DNN, all nodes in the current layer are combined, and each node has an individual weight factor. They are connected to the nodes of the next layer. However, in the CNN structure, the number of connections is limited by using a predetermined size of the filters, and hence, we can greatly reduce the amount of computation, compared to the DNN structure. The filters are updated as the learning process progresses. In general, the number of filters is adjusted according to the degree of feature extraction required in image processing models, and the computation needed increases as the number of filters increases. In our proposed models, because the input data are structured in a single dimension, we use a single filter to increase the learning efficiency of the models.

If data are processed through the filter, existing information is amplified, ignored, or reduced according to its characteristics. As a result of such processing, emphasized characteristic data and noise data are mixed. When the number of nodes and related parameters increases during learning, the learning speed and effect decrease. To prevent such degradation, a pooling step is necessary for the CNN model. In general, max-pooling that extracts the strongest signal (value) is popular, and hence, this study also uses max-pooling to prevent the deterioration of learning speed and level.

The general CNN model correlates only with adjacent nodes and analyzes the results. Due to this restrictive correlation, a CNN has not been used for studies where all correlations should be considered. To overcome this limit, this study proposes a fully connected expanding structure that is applied before entry into the CNN. By adding an expanding layer, the model not only considers all possible correlations between nodes but can also reduce the learning complexity.

4.3.2.1 CNN 1-dimensional model architecture

In this study, two CNN models are proposed. The first is a CNN model with a one-dimensional data processing structure (CNN1d). The expanding layer of the CNN1d model has 2,048 nodes, and the analysis structure has two convolutions with a filter size of 32. Also, it has a single max-pooling step with a size of 2×2 . The convolution layer receives nodes as the size of the filter and makes them into one value by a convolution

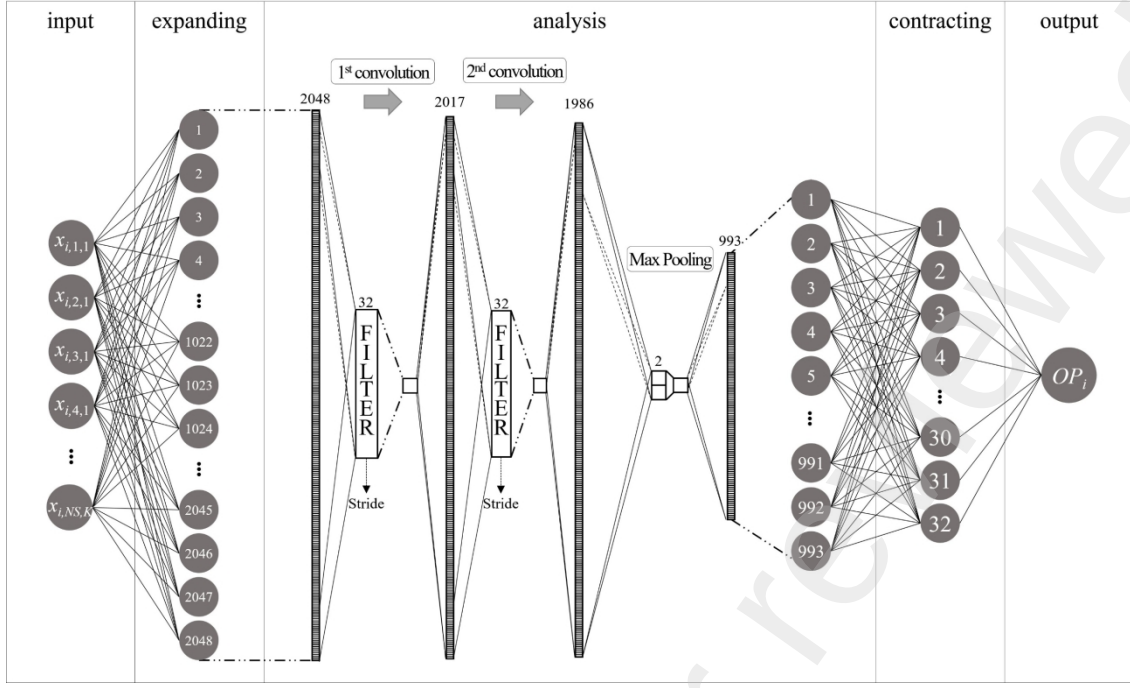


Fig. 7: A graphical example of the CNN1d model

product. The obtained value is sent to a node in the next layer. Then, the same operation is repeated by moving them at a predetermined stride ($= 1$). Because a filter operation for 32 nodes returns a single value, the number of nodes decreases according to the size of the filter. The number of nodes generated through the convolution layer can be obtained with Equation (3) below. The max-pooling layer extracts nodes sized 2×2 , and it sends the largest value to the node in the next layer. The structure of the suggested CNN1d is shown in Fig. 7.

$$node_{output} = \frac{node_{input} + 2(padding) - filter\ size}{stride} + 1 \quad (3)$$

4.3.2.2 CNN 3-dimensional model architecture

In the proposed CNN3d model, two convolutions with 2,197 nodes in the expanding layer and two filters sized $4 \times 4 \times 4$ and $3 \times 3 \times 3$ are applied to the analysis structure. The 2,197 nodes in a single dimension are matched sequentially to nodes in a cube at $13 \times 13 \times 13$ in the analysis layer. For example, the first and the second nodes of the expanding layer are placed $(1, 1, 1)$ and $(1, 1, 2)$ in the first cube of $13 \times 13 \times 13$. It has one max-pooling structure with an input size of $(2 \times 2 \times 2)$. Fig. 8 shows the structure of the CNN3d model. As seen in the figure, the filter of size $4 \times 4 \times 4$ is used in the first convolution layer and the filter of size $3 \times 3 \times 3$ is used in the next layer. Since the purpose of the CNN3d model is to obtain a single value (OP_i), it is natural to reduce the size of cubes in the analysis structure. Therefore, we use the large filter first and the small one later to reduce the cube size, so it converges effectively to a single value.

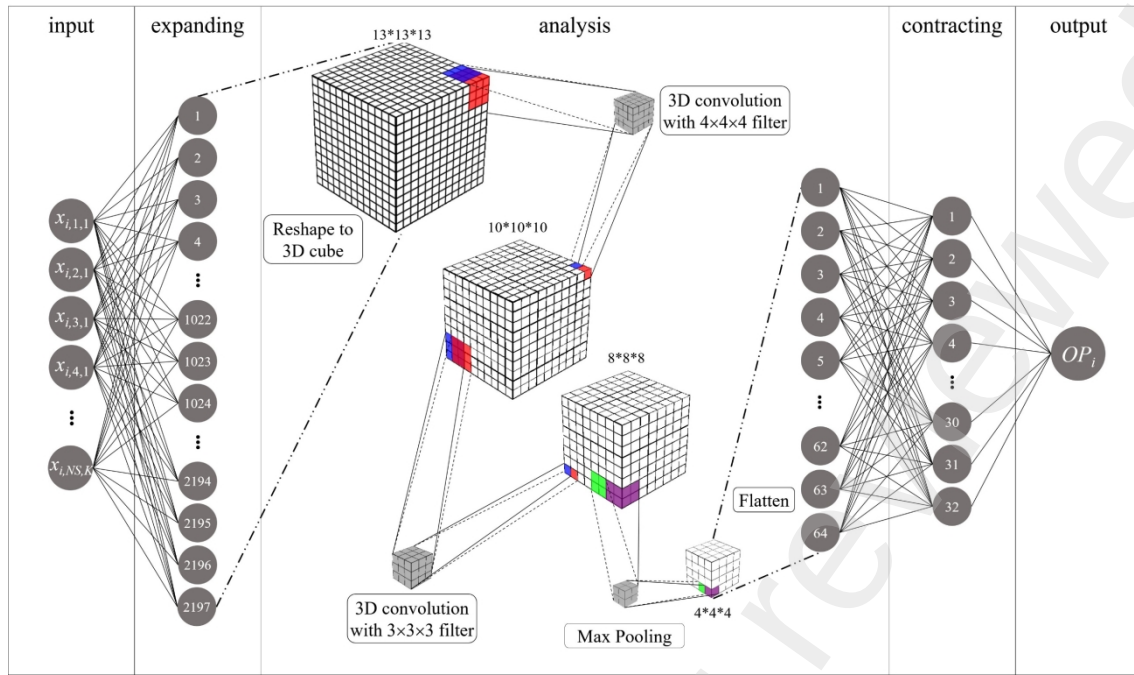


Fig. 8. A graphical example of the CNN3d model

4.4 Overall procedure for dispatching lots

This subsection provides a procedure for training the proposed machine learning models. The models estimate OP_i values for all possible recipe combinations. The overall procedure for training machine learning models by using generated data is given below.

Procedure 1 (Training Machine Learning Models)

- Step 1:** Collect the history logs of the recipe processes from the wet clean station.
- Step 2:** Considering the size of the set (NS), generate all possible sets from the history logs.
- Step 3:** For each set i , convert the sequence of the recipes in the set to an input data structure with binary variables (x_{ijk}) for the ML models (see Subsection 4.1).
- Step 4:** ML models learn the training data and obtain a trained model to estimate an OP_i value for each set i .
- Step 5:** Estimate the OP_i values for all possible sets using the trained model.

Assume there are four lots in the wet clean station. Then, we can make set i with $(NS - 1)$ recipes that were recently put in the wet clean station and a new recipe to be dispatched. For set i , we estimate the OP_i value with ML models. That is, the new recipe determines the OP_i value. This estimation is performed on all possible sets, and the estimated OP_i value becomes a criterion for dispatching the next recipe to the wet clean station. That is, the possible sets are sorted in descending order of OP_i values. Therefore, a dispatcher selects the next recipe that gives the largest OP_i from among the candidate recipes.

5. Computational Results

In this section, we report the results from the evaluation of the proposed ML models: MLR, DNN, and two CNN models. To examine the models, we compared them with two simple algorithms (random selection and same-recipe selection) that are used in a semiconductor manufacturing company in the Republic of Korea. All models were implemented in the Python programming language, and all experiments were performed using the Keras library on a PC with an Intel Core i5-9600 CPU and 8GB RAM. To obtain learning data for training the ML models, we developed a simulator in Python.

5.1 Data Generation

Machine learning models require a lot of data for training. However, actual history logs could not be obtained from the company owing to confidentiality. Instead of the real data, we developed a simulation model based on the actual processes of the company. In the simulation model, a wet clean station has eight baths arranged sequentially, and the immersion time in each bath was generated randomly from a uniform distribution with a range [1, 9] seconds, considering actual manufacturing situations. It assumed that all lots are waiting to enter the wet clean station at the start of the simulation.

We considered 5 scenarios with different patterns of bath usage. The detailed scenarios are given in Table 1, which shows the 10 recipes in each scenario. Scenario sc1 considers a situation where all recipes use two types of baths. Also, sc2 and sc3 assume that all recipes use four and six baths, respectively, while the numbers of baths for the recipes in sc4 are mixed. The baths used in recipes were randomly selected. On the other hand, in sc5, all recipes must visit bath 8 which is the last one in the wet station. In the actual fab, some wet clean stations have a water bath or air dryer placed at the last position, and recipes processed in such stations are always cleaned in the water bath or air dryer at last. In each scenario, the simulator started with 1,000 lots for each recipe, i.e., a total of 10,000 lots, which is approximately equivalent to two weeks of workloads in an actual situation. By running the simulator, history logs of 10,000 lots in each scenario were generated, and they were divided as follows: 8,100 for training, 900 for validation, and 1,000 for testing. In training, the models described in Section 4 were trained for input length and scenarios. The validation data were not involved in training but were used to prevent overfitting while learning. Considering four ML models with four levels of NS (=

Table 1: Bath usage information in the 5 scenarios

recipe	sc1	sc2	sc3	sc4	sc5
1	1, 3	1, 2, 3, 8	1, 2, 3, 4, 5, 6	1, 2, 3, 5, 6, 8	1, 2, 4, 8
2	1, 4	1, 3, 5, 8	1, 2, 3, 5, 6, 7	1, 2, 4, 5, 6, 7	1, 2, 6, 8
3	1, 7	1, 4, 6, 7	1, 2, 3, 6, 7, 8	1, 2, 4, 5, 7, 8	1, 3, 6, 8
4	2, 6	2, 3, 4, 5	1, 2, 4, 5, 6, 7	1, 3, 4, 5, 7, 8	1, 3, 7, 8
5	2, 8	2, 3, 4, 8	1, 2, 4, 5, 6, 8	1, 4	1, 6, 7, 8
6	2, 8	2, 3, 5, 8	1, 2, 5, 6, 7, 8	1, 6	2, 3, 5, 8
7	4, 5	2, 3, 5, 8	2, 3, 4, 5, 6, 7	2, 3, 5, 8	2, 3, 6, 8
8	4, 7	2, 3, 7, 8	2, 4, 5, 6, 7, 8	3, 4	2, 4, 6, 8
9	5, 8	2, 4, 5, 7	3, 4, 5, 6, 7, 8	5, 6, 7, 8	3, 6, 7, 8
10	5, 8	2, 4, 5, 8	3, 4, 5, 6, 7, 8	5, 7	5, 6, 7, 8

4, 5, 6, and 7) for 5 process scenarios, 80 models were created, and training was carried out in 10 epochs for each model.

To train the ML models, we defined the data for input and output. After completing the simulations, we could obtain a record of the lots and recipes tracked into and out of the wet clean station. Then, sets at size NS were made for all possible combinations, and the recipe sequence of each set was transformed into the input data structure for the ML models, as explained in Subsection 4.1. All ML models yielded the estimated OP_i value for the input data of set i .

5.2 Results Analysis and Discussion

The performances of the proposed ML models were summarized and analyzed. The equipment considered in this study can process up to four lots in the station. Therefore, to reflect the correlation between lots, we tested four levels of NS ($= 4, 5, 6$, and 7) to change the amount of learning. To verify these results, scatter plots are presented in Fig. 9. The values on the y -axis represent the predicted OP values from the test data, while the values on the x -axis represent the OP values obtained from the simulation results. For example, in cases of $NS4$, the predicted and simulated OP values for each set of four wafer lots loaded into the wet clean station in succession were plotted on the y -axis and x -axis, respectively. Note that, if the scatter plot lies near the $y = x$ diagonal, it indicates the learning has been done ideally without any overfitting. Otherwise, substantial deviations from the diagonal present possible evidence of overfitting in the learning. As shown in Fig. 9, the plots of MLR model were highly dispersed from the $y = x$ diagonal, while those of the other models show a linear fit with little variation around the diagonal. That means a single-layer model is limited in explaining the problem, and hence, multi-layered models are more suitable. In addition, the CNN1d and CNN3d models show very similar distributions in all ranges. Focusing on the plots for sc3 and

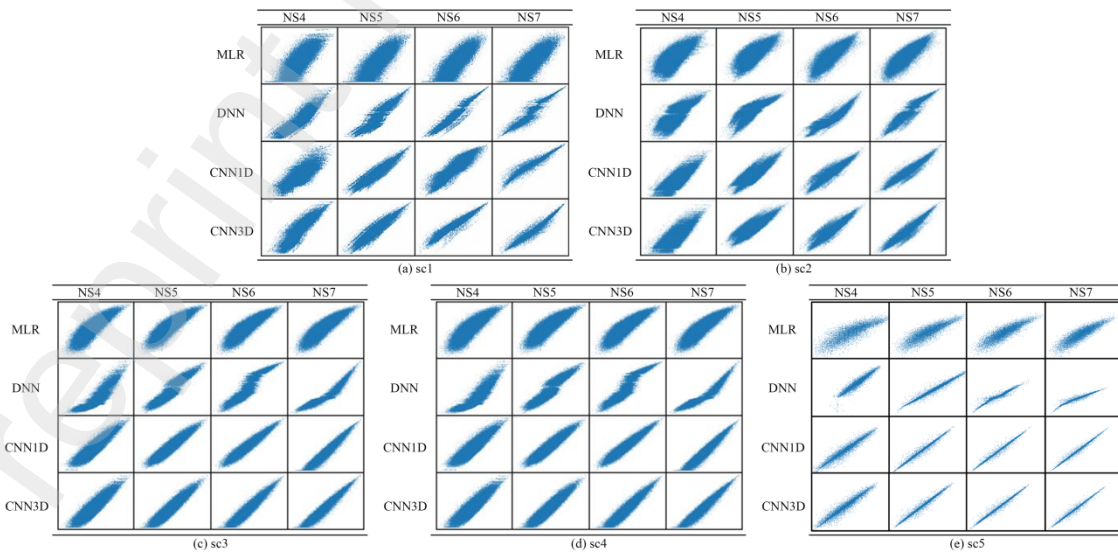


Fig. 9: Scatter plots of four models and four values of NS in (a) sc1, (b) sc2, (c) sc3, (d) sc4, and (e) sc5

sc4, in which more baths are used, the convergence of the CNN-based models appears more clearly than for the MLR and the DNN. Therefore, it can be inferred that CNN models predict better as the use of baths increases. In the case of sc5, the plots show a linear close to the diagonal, which is clearer than in sc4. It means that our suggested models can work well when using a certain bath in all (or most) recipes.

5.2.1 Single-lot dispatching

The proposed ML models can predict wet clean station utilization depending on which recipe will be selected next. Thus, using the ML models, we can choose the best next recipe by considering the recipes already in progress inside the station. With this selection method, a complete sequence can be obtained by repeatedly selecting the next recipe one by one. We call this method *single-lot dispatching*. To evaluate this method, we compared it with two methods (same-recipe selection and random selection) that are used in an actual fab. Same-recipe selection continues to take a recipe identical to the one that just entered the wet clean station, whereas random selection chooses the next recipe at random. Note that same-recipe selection is almost equal to SPT or LPT because the same recipe has the same estimated processing time. Also, in an actual semiconductor fab, FCFS is commonly used. However, since it assumes that all lots are ready at time zero, we used the random selection method for comparison instead of FCFS.

Table 2 summarizes the makespan values obtained by sequencing the 10,000 lots using the proposed ML models, same-recipe selection, and random selection with four different values for *NS*. Note that same-recipe selection and random selection are not ML, and do not use *NS*. As seen in the table, the proposed ML-based dispatching rules obtained

Table 2: Makespan values of the suggested model-based dispatching rules

<i>NS</i>	Model	sc1	sc2	sc3	sc4	sc5
-	Same	61298.5	73329.7	85964.2	79147.7	69129.5
	Random	53402.7	85284.4	111169.1	107210.9	74165.8
4	MLR	61284.1	73325.6	85970.5	79605.9	69152.8
	DNN	34607.0	68877.7	86224.7	71264.5	68067.8
	CNN1d	34109.4	70720.9	87086.1	72031.2	65350.1
	CNN3d	38095.4	69520.6	85969.0	69049.9	67910.0
5	MLR	61284.6	73325.3	85987.6	79514.2	69151.8
	DNN	42797.1	68321.1	87345.1	68032.3	68100.0
	CNN1d	31327.9	72516.2	85986.8	73843.6	61864.4
	CNN3d	41436.3	70720.9	87949.4	74765.3	61852.8
6	MLR	61284.7	73318.7	85972.5	79332.9	69152.0
	DNN	35644.0	69515.9	85974.6	72390.9	72928.9
	CNN1d	42691.3	69932.0	86981.1	75687.0	63457.7
	CNN3d	39035.1	69519.4	86896.8	71562.4	57499.1
7	MLR	61278.6	73318.6	85972.5	78934.9	69148.8
	DNN	37181.4	69955.5	85983.6	70239.2	70622.9
	CNN1d	37259.3	69847.3	85983.8	72394.4	58084.7
	CNN3d	41136.1	69530.7	86568.7	71823.4	66854.8

better sequences than the two simple methods. In sc1, the random selection method gave better performance than the same-recipe method, which means that with a small number of baths, the same-recipe method may not be efficient. Therefore, sequencing the recipes is important to reduce makespan. We find this feature in the computational results. That is, the DNN and CNN1d models showed better results in sc1, reducing makespan by 35.2% and 36.1%, respectively, compared to the random selection method. As NS increased, the input data contained more information about preceding recipes, and hence, the DNN accordingly constructed more nodes and worked better. In sc2, the DNN model showed overall excellent performance. However, the makespan values obtained in sc3 were very similar, excluding those from the random selection method. This may be because AP values of different recipe combinations were similar in sc3, where every recipe used almost all the baths. Thus, the same-recipe method also showed good performance in sc3. In other words, the suggested ML models can work better when the variance in AP s is large. This can be confirmed from the results for sc4 in which sc1, sc2, and sc3 were mixed. That is, the DNN and CNN3d showed good performance in sc4. In cases of sc5, CNN1d and CNN3d give good solutions compared to DNN and MLR. CNN1d reduced 21.7% when $NS=7$, which is the largest size of the input data. Even though sc5 has an additional constraint for using baths, all suggested algorithms give better solutions than random selection and same recipe policies. The total computation time for dispatching 10,000 lots (except for the learning time of each model) was about two seconds, which is good enough for the real world. The performance of sc5 is the best. In an actual fab, various scenarios are mixed as in sc4, or most recipes use a certain bath (or baths) as in sc5, so our proposed ML-based dispatching methods can be applied and will help enhance the productivity of the wet clean station.

Table 3 shows the number of parameters used for training in the high-dimensional solve models. In general, learning and calculation times depend on the number of parameters, and time increases as the number of parameters increases. As can be seen from the table, compared to the DNN, CNN1d, and CNN3d used only 10.2% and 8.7% of the parameters, respectively. Reduction of parameters can have the effect of reducing learning time and computational load. Therefore, the results in Table III can be used to select a suitable model considering the computational load when using the model in an actual fab. CNN models showed similar performance using a small number of parameters, compared to the DNN model. Consequently, CNN models can be effective methods if the computational capabilities of the computers are insufficient in small and medium-sized companies.

Table 3: Number of parameters used in the high-dimensional solve models

NS	DNN	CNN1d	CNN3d
4	1,412,161	115,875	92,283
5	1,432,641	136,355	114,253
6	1,453,121	156,835	136,223
7	1,473,601	177,315	158,193
Average	1,442,881	146,595	125,238

5.2.2 Multiple-lot dispatching

The models proposed in this paper select the best recipe combination at the dispatching time. For the wet clean station considered in this study, up to three lots can be dispatched at a time because the four lots can be processed simultaneously. If multiple lots are dispatched at a time, the required time for sequencing numerous lots can be reduced. Note that with the process mentioned above, it is theoretically possible to determine the sequence of all lots at once, but in real-world situations, variations occur in the production process, so the number of lots dispatched at one time is limited. Fig. 10 shows examples of single-lot dispatching and multiple-lot dispatching (three lots) when $NS=4$. That is, the single-lot dispatching selects one lot at a time (Fig. 10-(a)), while the multiple-lot dispatching method makes a sequence by selecting the next three lots in order (Fig. 10-(b)).

Fig. 11 presents makespan values of the proposed ML models with different NS values in sc5. In the figure, Pick X means the number of lots dispatched at one time. In the MLR model, makespan increases as X increases, while this tendency does not appear in the high-dimensional solve models. Thus, we can construct a schedule with the multiple-lot dispatching method, and it can help planners or operators make plans for the production or maintenance of the wet clean station. The given schedule also makes it possible to predict and prepare for what will happen next. Note that the computation times of the multiple-lot dispatching method with Pick 2 and 3 were about 5 and 33 seconds, respectively. Although they are quite different, these times for sequencing 10,000 lots are acceptable in an actual fab.

6. Concluding Remarks

This study addressed the scheduling problem of a wet clean station in semiconductor manufacturing, where information on the internal operations and settings is not available. As typical scheduling or dispatching algorithms cannot be used for the problem due to the unavailability of information, this study proposed machine learning-

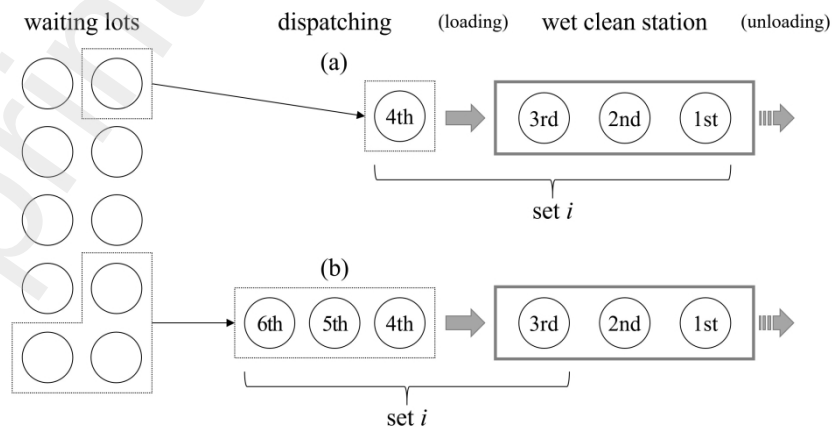


Fig. 10: Examples of (a) single-lot dispatching and (b) multiple-lot dispatching when $NS = 4$

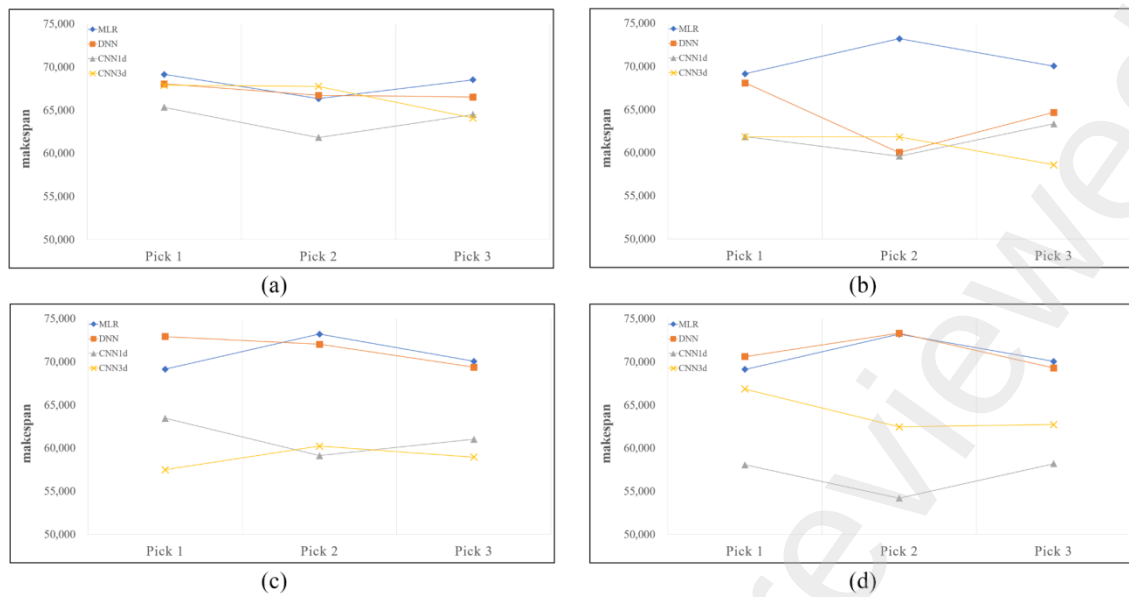


Fig. 11: Makespan of ML models with different NS values for $sc5$:
(a) $NS=4$, (b) $NS=5$, (c) $NS=6$, and (d) $NS=7$

based dispatching methods to find the best next recipe with the objective of minimizing the makespan. The development of the ML models was based only on the track in and out times of each wafer lot. This study also suggested a method of converting a sequence of recipes into input data in a matrix structure. The proposed ML models successfully predicted the times that wafer lots spend in the wet clean station, depending on the combination of recipes sequentially processed in the station. Using the prediction results, we found the best recipe combinations and our finding makes it possible to dispatch the best next recipe and to construct a complete schedule for the wet clean station. Through a series of computational tests with various scenarios, we evaluated the proposed models by comparing them with two methods used in an actual fab in Korea. The results confirmed the applicability of the proposed models to an actual fab and demonstrated that the models can improve the productivity of the wet clean station.

Through this study, we confirmed the possibility of developing ML models for sequencing (or dispatching) lots in an environment where there is no progress history inside the facility. Also, we expect the proposed models can be used not only for wet clean station equipment but also for other semiconductor processes, such as implantation and CMP, in which processing data inside the machine is unknown. Furthermore, it is expected that the suggested models can be used to improve the productivity of equipment that processes various products. However, this study limited the types of products to 10. Thus, if more diverse products need to be considered, learning time should also be increased. Therefore, it may be necessary to enhance the suggested models by constructing better structures for input data and layers.

References

- Baly, R., & Hajj, H. (2012). Wafer classification using support vector machines. *IEEE Transactions on Semiconductor Manufacturing*, 25(3), 373-383. <https://doi.org/10.1109/TSM.2012.2196058>.
- Benardos, P. G., & Vosniakos, G. C. (2007). Optimizing feedforward artificial neural network architecture. *Engineering Applications of Artificial Intelligence*, 20(3), 365-382. <https://doi.org/10.1016/j.engappai.2006.06.005>.
- Bergmann, S., Feldkamp, N., & Strassburger, S., (2016). Emulation of control strategies through machine learning in manufacturing simulations. *Journal of Simulation*, 11(1), 38-50. <https://doi.org/10.1057/s41273-016-0006-0>.
- Chung, T., Gupta, J. N. D., Zhao, H., & Werner, F. (2019). Minimizing the makespan on two identical parallel machines with mold constraints. *Computers & Operations Research*, 105, 141-155. <https://doi.org/10.1016/j.cor.2019.01.005>.
- Cho, Y. I., Nam, S. H., Cho, K. Y., Yoon, H. C., & Woo, J. H. (2022). Minimizing makespan of permutation flowshop using a pointer network. *Journal of Computational Design and Engineering*, 9(1), 51-67. <https://doi.org/10.1093/jcde/qwab068>.
- Florian, E., Sgarbossa, F., & Zennaro, I. (2021). Machine learning-based predictive maintenance: A cost-oriented model for implementation. *International Journal of Production Economics*, 236, 108114. <https://doi.org/10.1016/j.ijpe.2021.108114>.
- Fukushima, K. (1988). Neocognitron: A hierarchical neural network capable of visual pattern recognition. *Neural Networks*, 1(2), 119-130. [https://doi.org/10.1016/0893-6080\(88\)90014-7](https://doi.org/10.1016/0893-6080(88)90014-7).
- Garey, M. R., Johnson, D. S., & Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2), 117-129. <https://doi.org/10.1287/moor.1.2.117>.
- Govind, N., Bullock, E. W., He, L., Lyer, B., Krishna, M., & Lockwood, C. S. (2008). Operations management in automated semiconductor manufacturing with integrated targeting, near real-time scheduling, and dispatching, *IEEE Transactions on Semiconductor Manufacturing*, 21(3), 363-370. <https://doi.org/10.1109/TSM.2008.2001211>.

- Graham, R. L., Lawler, E. L., Lenstra, J. K., & Kan, A. H. G. R. (1979). Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*, 5, 287-326. [https://doi.org/10.1016/S0167-5060\(08\)70356-X](https://doi.org/10.1016/S0167-5060(08)70356-X).
- Han, A. M., & Cho, A. M. (2015). A practical two-phase approach to scheduling of photolithography production. *IEEE Transactions on Semiconductor Manufacturing*, 28(3), 367-373. <https://doi.org/10.1109/TSM.2015.2451512>.
- Jeong, B. J., Han, J. H., & Lee, J. Y. (2021). Metaheuristics for a flow shop scheduling problem with urgent jobs and waiting times. *Algorithms*, 14(11), 323. <https://doi.org/10.3390/a14110323>.
- Johri, P. K. (1989). Optimality of the shortest line discipline with state-dependent service rates. *European Journal of Operational Research*, 41(2), 157-161. [https://doi.org/10.1016/0377-2217\(89\)90378-0](https://doi.org/10.1016/0377-2217(89)90378-0).
- Jung, C., Kim, H. J., & Lee, T. E. (2013). A branch and bound algorithm for cyclic scheduling of timed Petri nets. *IEEE Transactions on Automation Science and Engineering*, 12(1), 309-323. <https://doi.org/10.1109/TASE.2013.2285221>.
- Kim, H. J., Lee, J. H., & Lee, T. E. (2014). Non-cyclic scheduling of a wet station. *IEEE Transactions on Automation Science and Engineering*, 11(4), 1262-1274. <https://doi.org/10.1109/TASE.2013.2255037>.
- Kim, H. J., Lee, J. H., & Lee, T. E. (2015). Noncyclic scheduling of cluster tools with a branch and bound algorithm. *IEEE Transactions on Automation Science and Engineering*, 12(2), 690-700. <https://doi.org/10.1109/TASE.2013.2293552>.
- Kim, Y. D., Bang, J. Y., An, K. Y., & Lim, S. K. (2008). A due-date-based algorithm for lot-order assignment in a semiconductor wafer fabrication facility. *IEEE Transactions on Semiconductor Manufacturing*, 21(2), 209-216. <https://doi.org/10.1109/TSM.2008.2000261>.
- Klemmt, A. & Monch, L. (2009). Scheduling jobs with time constraints between consecutive process steps in semiconductor manufacturing. *In Proceedings of the 2012 Winter Simulation Conference*, Austin, TX, USA, 2173- 2182. <https://doi.org/10.1109/WSC.2009.5429171>.

- Lee, J. H., & Kim, H. J. (2022). Reinforcement learning for robotic flow shop scheduling with processing time variations, *International Journal of Production Research*, 60(7), 2346-2368. <https://doi.org/10.1080/00207543.2021.1887533>.
- Lee, T. E., Lee, H. Y., & Lee, S. J. (2007). Scheduling a wet station for wafer cleaning with multiple job flows and multiple wafer-handling robots. *International Journal of Production Research*, 45(3), 487-507. <https://doi.org/10.1080/00207540600792531>.
- Lee, T. E. (2008). A review of scheduling theory & methods for semiconductor manufacturing cluster tools. *Winter Simulation Conference*. Miami, USA, 2127-2135. <https://doi.org/10.1109/WSC.2008.4736310>.
- Lin, C. C., Deng, D. J., Chih, Y. L., & Chiu, H. T. (2019). Smart manufacturing scheduling with edge computing using multiclass deep Q network. *IEEE Transactions on Industrial Informatics*, 15(7), 4276-4284. <https://doi.org/10.1109/TII.2019.2908210>.
- Oh, S., Cho, Y. I., & Woo, J. H. (2022). Distributional reinforcement learning with independent learners for the flexible job shop scheduling problem with high variability. *Journal of Computational Design and Engineering*, 9(4), 1157-1174. <https://doi.org/10.1093/jcde/qwac044>.
- Pang, J., Zhou, H., Tsai, Y. C., & Chou, F. D. (2018). A scatter simulated annealing algorithm for the bi-objective scheduling problem for the wet station of semiconductor manufacturing. *Computers & Industrial Engineering*, 123, 54-66. <https://doi.org/10.1016/j.cie.2018.06.017>.
- Poriore, P., Fuente, D. D. L., Puente, J., & Parreno, J. A comparison of machine-learning algorithms for dynamic scheduling of flexible manufacturing systems. *Engineering Applications of Artificial Intelligence*, 19(3), 247-255, Apr, 2006. <https://doi.org/10.1016/j.engappai.2005.09.009>.
- Scholl, W. & Domschke, J. (2000). Implementation of modeling and simulation in semiconductor wafer fabrication with time constraints between wet etch and furnace operations. *IEEE Transactions on Semiconductor Manufacturing*, 13(3), 273-227. <https://doi.org/10.1109/66.857935>.

- Shahrabi, J., Adibi, M. A. & Mahootchi, M. (2017). A reinforcement learning approach to parameter estimation in dynamic job shop scheduling, *Computers & Industrial Engineering*, 110, 75-82. <https://doi.org/10.1016/j.cie.2017.05.026>.
- Shiue, Y. R., Lee, K. C., & Su, C. T. (2018). Real-time scheduling for a smart factory using a reinforcement learning approach. *Computers & Industrial Engineering*, 125, 604-614. <https://doi.org/10.1016/j.cie.2018.03.039>.
- Susto, G. A., Schirru, A., & Pampuri, S. (2014). Machine learning for predictive maintenance: A multiple classifier approach. *IEEE Transactions on Industrial Informatics*, 11(3), 812-820. <https://doi.org/10.1109/TII.2014.2349359>.
- Wang, Y. C., & Usher, J. M. (2005). Application of reinforcement learning for agent-based production scheduling. *Engineering Applications of Artificial Intelligence*, 18(1), 73-82. <https://doi.org/10.1016/j.engappai.2004.08.018>.
- Wein, L. M. (1988). Scheduling semiconductor wafer fabrication. *IEEE Transactions on Semiconductor Manufacturing*, 1(3), 115-130. <https://doi.org/10.1109/66.4384>.
- Wu, T., Huang, E., Wang, M., & Zheng, M. (2022). Job scheduling of diffusion furnaces in semiconductor fabrication facilities. *European Journal of Operational Research*, vol. 301, no. 1, 141-152. <https://doi.org/10.1016/j.ejor.2021.09.044>.
- Yu, T. S., Kim, H. J., & Lee, T. E. (2018). Scheduling single-armed cluster tools with chamber cleaning operations. *IEEE Transactions on Automation Science and Engineering*, 15(2), 705-716. <https://doi.org/10.1109/TASE.2017.2682271>.
- Yu, T. S. & Han, J. H. (2021). Scheduling proportionate flow shops with preventive machine maintenance. *International Journal of Production Economics*, 231, 107874. <https://doi.org/10.1023/A:1008904604531>.
- Yurtsever, T. E., Kutanoglu, & Johns, J. (2009). Heuristic based scheduling system for diffusion in semiconductor manufacturing. In *Proceedings of the 2009 Winter Simulation Conference*, Austin, TX, USA, 1677- 1685. <https://doi.org/10.1109/WSC.2009.5429171>.